5-Day Intensive Roadmap

LangChain, LangGraph & Agentic AI — Capstone Week

Mon Aug 17 through Fri Aug 21, 2026 (continues from last week: Gen AI fundamentals, prompt engineering, LLM APIs, embeddings & vector search)

How this plan works

Last week you worked directly against the raw API and NumPy: you called the model, engineered prompts by hand, wrote tool-calling loops yourself, and built a semantic search index from scratch with Chroma or FAISS. That was deliberate — you needed to see what the frameworks abstract away before you trust them.

This week you move to frameworks that do this for production systems: LangChain for the building blocks, LangGraph for stateful multi-step agents. By Friday you ship a single agentic RAG service that reuses your Project 2 database and auth. This is a demanding week — treat every day as a full 8-hour work day, not a tutorial to skim. If a concept from last week is shaky, fix it before moving on; everything here builds directly on tokens, context windows, prompting, tool calling, and embeddings.

Rule for the week: nothing is 'done' until it runs, you can explain why it works, and you can answer the theoretical questions without looking at your code.

Week at a glance

Day	Date	Focus
Day 1	Mon, Aug 17	LangChain core: LCEL, chains, retrievers, structured output — rebuild last week's RAG pipeline properly
Day 2	Tue, Aug 18	Memory, tools, and the LangChain agent abstraction
Day 3	Wed, Aug 19	LangGraph: stateful graphs, cycles, checkpointing, human-in-the-loop
Day 4	Thu, Aug 20	Agentic AI patterns: reflection, planning, guardrails, evaluation
Day 5	Fri, Aug 21	CAPSTONE (full day): agentic RAG assistant — API + DB + agent + guardrails + deploy

Day 1 (Mon, Aug 17): LangChain Core & RAG Pipelines

Goal: stop hand-rolling chains. Learn LCEL well enough to compose retrieval, prompting, and parsing into one declarative pipeline, and understand exactly what each piece replaces from last week's manual code.

Theory & Concepts

- LangChain architecture: models, prompt templates, and output parsers as interchangeable Runnable components.
- LCEL (LangChain Expression Language): the pipe operator, the Runnable interface, and why it gives you streaming, batching, and async for free.
- RunnableParallel and RunnablePassthrough for feeding multiple inputs (e.g. retrieved context and the original question) into one prompt.
- Document loaders (PDF, web, plain text) and text splitters — RecursiveCharacterTextSplitter, chunk size vs overlap trade-offs.
- Embeddings and vector store wrappers (Chroma) inside LangChain, versus the raw API calls you wrote on Day 4 last week.
- Retrievers: similarity search, MMR (maximal marginal relevance), score thresholds, and a retriever as a Runnable.
- Document combination strategies: stuff, map_reduce, refine — what each costs in tokens and latency.
- Structured output: Pydantic-backed output parsers, and why 'ask for JSON' alone is not enough in production.

Theoretical Questions (answer in writing, no code)

1. Explain why the Runnable interface makes chains composable and natively streamable, compared to chaining plain Python function calls.
2. Compare 'stuff', 'map_reduce', and 'refine' document combination strategies. When is map_reduce unavoidable, and what does it cost you in latency and token spend?
3. Why does chunk overlap reduce boundary-cutoff problems in retrieval, and why does increasing overlap past a point stop helping and start hurting (cost, redundancy)?
4. Explain MMR in your own words and describe a query where you would prefer it over plain top-k similarity search.
5. You ask a model for JSON and validate it with a Pydantic model. The validation fails on 1 in 20 calls. List three different fixes at three different layers of the system (prompt, parsing, application) and the trade-off of each.

Coding Questions (write and run real code)

6. Build an LCEL chain: prompt | model | StrOutputParser(). Then extend it with RunnableParallel so both a retrieved context and the original question flow into the prompt template in one composed chain.
7. Define a Pydantic schema for a structured extraction task (e.g. pull invoice number, vendor, and total from free text). Build a chain that parses model output against it, and add a retry loop that re-prompts with the validation error if parsing fails, up to 3 attempts.
8. Without using Chroma, implement your own minimal cosine-similarity retriever class exposing the same .invoke(query) interface as a LangChain retriever, backed by a plain Python list of (text, embedding) pairs and NumPy. This proves you know what the abstraction is hiding.
9. Bug hunt: you are given a RAG chain where the retriever always returns the same 2 chunks regardless of the query. Write the debugging checklist you would run through (in code, not just prose) to isolate whether the bug is in chunking, embedding, indexing, or the retriever call.

Exercise / Hands-on Task

Rebuild Day 4 last week's manual RAG pipeline end-to-end using LangChain LCEL. Time yourself. Write a 5-to-8 sentence note comparing code length, readability, and what you'd lose in debuggability by using the abstraction instead of the raw calls.

Docs: *python.langchain.com, python.langchain.com/docs/concepts/lcel*

Day 2 (Tue, Aug 18): Memory, Tools & the Agent Abstraction

Goal: give the model access to real actions and multi-turn state, and understand exactly where LangChain's built-in agent abstraction starts to break down — which is the reason Day 3 exists.

Theory & Concepts

- Conversation memory patterns: buffer memory, summary memory, and windowed memory — why raw unbounded history silently degrades output quality instead of erroring.
- Defining tools: name, description, and a typed args schema — why the description text is what the model actually reasons over when choosing a tool.
- Native tool-calling agents vs older text-parsing ReAct agents that ask the model to emit 'Action: ...' strings.
- AgentExecutor: the plan-act-observe loop LangChain runs for you, and its default iteration and error handling.
- Handling a tool that raises an exception mid-run without crashing the whole agent.
- Multiple tools and disambiguation: what happens when two tools could plausibly answer the same query.
- Where LangChain's built-in agent loop stops being enough: no easy branching, no persistence, no human approval step — the motivation for LangGraph.

Theoretical Questions (answer in writing, no code)

10. Why can memory silently degrade quality without ever throwing an error? How would you detect this happening in production rather than by eyeballing outputs?
11. Compare a native tool-calling agent with a text-parsing ReAct agent. Why is native tool-calling more reliable, and in what situation would the older approach still be defensible?
12. An agent has 'search_database' and 'search_documents' tools and keeps picking the wrong one for ambiguous queries. Walk through, step by step, how you'd diagnose whether this is a description problem, a tool-count problem, or a prompt problem — and how you'd fix each.
13. What specifically can a LangGraph state machine express that a plain AgentExecutor loop cannot? Name two concrete capabilities.

Coding Questions (write and run real code)

14. Define three tools with the @tool decorator: a calculator, a read-only SQL lookup against your Project 2 database, and a stub web-search function. Give each a strict typed args schema, not just a free-text string argument.
15. Build a tool-calling AgentExecutor and run a query that requires two tools in sequence (e.g. look up a post by id, then compute something about its comment count). Log every intermediate tool call and its result.
16. Make one tool raise an exception on a specific input. Catch it inside the tool boundary so the agent reports a clean failure message to the user instead of crashing the process.
17. Wrap Day 1's RAG chain as a tool named search_my_docs and give the agent both that tool and the calculator tool. Ask it three questions: one that needs RAG, one that needs the calculator, one that needs neither. Confirm correct routing for all three.

Exercise / Hands-on Task

Extend the agent from the coding questions with summary memory so it can answer a follow-up question that depends on something said two turns earlier. Prove it works with a 4-turn conversation transcript.

Docs: *python.langchain.com/docs/concepts/tools, python.langchain.com/docs/concepts/agents*

Day 3 (Wed, Aug 19): LangGraph — Stateful, Graph-Based Agents

Goal: express agents as graphs so you can branch, cycle, pause, resume, and require human approval — none of which a linear AgentExecutor loop does cleanly.

Theory & Concepts

- Why graphs: modeling cyclic, stateful, multi-step workflows that a linear chain or loop can't express well.
- StateGraph: a shared state schema (TypedDict or Pydantic model), nodes as plain functions that read and update state, and edges connecting them.
- Conditional edges: routing to different next nodes based on the current state.
- Building the plan -> act -> observe -> reflect loop explicitly as a graph, instead of relying on a hidden default loop.
- Cycles and infinite-loop risk: why you must add an explicit max-iteration or termination guard yourself.
- Checkpointing and persistence: saving state at each node so a run can be paused and resumed by thread ID.
- Human-in-the-loop: interrupt nodes that pause execution until a human approves, and how that differs from just adding a confirmation prompt inside a tool.

Theoretical Questions (answer in writing, no code)

18. Explain why an infinite loop is possible in a LangGraph agent and describe the specific state field and guard you'd add to prevent it. Why doesn't a plain AgentExecutor need the identical fix?
19. What is a checkpoint in LangGraph, and how does persistence combined with a thread ID enable a pause-then-resume human-in-the-loop workflow days later?

20. Design, in words, the state schema for a multi-turn research agent that must remember prior tool results across graph steps without re-querying them. What fields does the state need, and what happens if a field is missing?
21. A conditional edge routes back to 'act' when the task looks incomplete. Describe a failure mode where this logic could loop forever even with a max-iteration guard in place, and how you'd catch it.

Coding Questions (write and run real code)

22. Build a StateGraph with three nodes — plan, act, observe — and one conditional edge that routes back to act or forward to an end node depending on whether the state marks the task complete.
23. Add an in-memory or SQLite checkpointer. Run the graph, stop it after the 'act' node on step 2, then resume it later using the same thread ID and confirm it continues from where it left off rather than restarting.
24. Add a human-in-the-loop interrupt immediately before any node that would call a delete-capable tool, so the graph pauses and requires an explicit approval value in state before continuing.
25. Convert Day 2's tool-calling agent into a LangGraph graph with an explicit max_iterations guard. Force one tool to always fail and show the graph terminates cleanly with a reported failure instead of looping.

Exercise / Hands-on Task

Take your Day 2 multi-tool agent, express it fully as a LangGraph graph with checkpointing, and write a short trace (state snapshots at each node) for one full run including one pause-and-resume cycle.

Docs: *langchain-ai.github.io/langgraph*

Day 4 (Thu, Aug 20): Agentic AI Patterns, Guardrails & Evaluation

Goal: make the agent trustworthy, not just functional — this is the day that separates a demo from something you'd let touch real data.

Theory & Concepts

- Agent patterns: ReAct, planning and task decomposition, reflection and self-correction, and multi-agent collaboration — what each buys you and what it costs in latency.
- Long-term memory (a vector store of past interactions) vs short-term memory (graph state) — when the agent genuinely needs to remember across sessions, not just within one.
- Guardrails: input validation, output validation against a schema, scoping which tools are callable and by whom, and hard limits on tool calls per request.
- Evaluating a non-deterministic system: building a fixed test set, defining what 'correct' means for open-ended output, and the specific risks of using an LLM as the judge.
- Handling hallucination and failure gracefully: fallback responses, bounded retries, and escalation to a human instead of guessing.

Theoretical Questions (answer in writing, no code)

26. What is the difference between reflection and planning in agent design? Can a single agent do both, and if so, sketch the order the steps would run in.
27. Why is 'LLM-as-judge' evaluation risky? Name two specific failure modes of this approach and one concrete mitigation for each.
28. Design three concrete guardrails for an agent that has write access to a production database. For each, state exactly what input or output it inspects and what it blocks.
29. An agent's reflection step approves its own bad answer 1 time in 10. Is adding a third reflection pass a good fix? Justify your answer with the trade-off it introduces.

Coding Questions (write and run real code)

30. Implement a self-reflection node: after the agent answers, a second LLM call critiques the answer against the original question and either approves it or sends it back for one more pass, capped at 2 retries total.
31. Add long-term memory: store a short summary of each conversation in a vector collection, and at the start of a new session retrieve and inject any relevant past summary before the first response.
32. Write a guardrail function that inspects a proposed SQL tool call and blocks any DELETE, DROP, UPDATE, or TRUNCATE statement before it reaches the database. Write a pytest suite for it covering at least 4 cases, including a case designed to bypass naive string matching (e.g. mixed case, comments, or whitespace tricks).
33. Add a hard cap of 5 tool calls per user request to your Day 3 graph, and show the exact state and message the user sees when the cap is hit mid-task.

Exercise / Hands-on Task

Take Day 3's graph and add both a reflection loop and the SQL guardrail. In your README, list the specific failure modes this configuration now catches that yesterday's version did not.

Docs: *langchain-ai.github.io/langgraph, docs.anthropic.com*

Day 5 (Fri, Aug 21): CAPSTONE — Agentic RAG Assistant (full 8-hour project day)

This is the hardest day of the internship so far. Budget the full day for it, work in feature branches with small commits, and do not skip the tests or the README — an undocumented, untested agent is not a finished project.

Build "DocOps Agent": a FastAPI service that wraps a LangGraph agent with retrieval, database, and reflection tools, reusing the auth and database from Project 2 (posts and comments API).

Required modules

34. Ingestion — an authenticated endpoint to upload a document; it is chunked, embedded, and stored in the vector store tagged with the owner's user_id from the JWT.
35. Retrieval tool — an agent tool performing semantic search scoped to only the authenticated user's own documents. Cross-user leakage is a failing bug, not an edge case.
36. Structured DB tool — a read-only agent tool that queries the Project 2 posts/comments tables through SQLAlchemy, exposed with a strict typed schema.
37. LangGraph orchestration — a plan/act/observe/reflect graph with conditional routing, an explicit max-iteration guard, and a checkpointer so a conversation thread can be resumed later.
38. Memory — short-term via graph state within a thread; long-term via a conversation_summaries vector collection consulted at the start of new threads.
39. Guardrails — block any destructive DB tool call, enforce per-user document scoping at the retrieval layer (not just in the prompt), cap tool calls per request, and validate every response against a Pydantic response schema before it leaves the API.
40. Auth — reuse Project 2's JWT auth; every agent action is scoped to the authenticated user, with no way to pass another user's id and read their data.
41. API — a single POST /agent/chat endpoint (streaming is a bonus, not required) plus GET /agent/history/{thread_id}.
42. Testing — a pytest suite covering: correct tool routing for at least 3 distinct query types, guardrail rejection of a destructive SQL attempt, cross-user document isolation, and one full end-to-end conversation asserting on the final state.
43. Documentation — a README with an architecture diagram (ASCII is fine), setup steps, and 3 example conversation transcripts each showing the agent choosing a different tool path.

Deliverable

A working service, runnable locally with docker-compose (API + Postgres + vector store), a Postman collection or demo script exercising all endpoints, and every requirement above green. Submit by end of day Friday and be ready to walk your Training Coordinator through a live demo, not a recording.

Tough bonus (optional, for the fastest finishers)

Implement a second 'reviewer' agent that independently checks every proposed SQL tool call before execution and can veto it, separate from the guardrail function. This is a real multi-agent collaboration pattern, not just a second guardrail — the reviewer should be its own LLM call reasoning about intent, not a string match.

Evaluation checklist your coordinator will run

- Ask a question answerable only from a document owned by a different test user — the agent must refuse or find nothing, never leak it.
- Ask the agent to delete or modify data through natural language — the guardrail must block it and the response must explain why.
- Kill the process mid-conversation and resume with the same thread_id — the conversation must continue coherently.
- Ask an ambiguous question requiring both the DB tool and the retrieval tool — check the agent chooses correctly and in a sensible order.
- Read the README cold, with no other context — can a new engineer run the project from it alone?